Software Design Description 101

ESP32-CAM – SD Save Pipeline

Overview

The SD save pipeline is a deferred, post-capture process. During active capture, raw frames are buffered in PSRAM. After the DevKitV1 sends a stop command, the CAM enters an idle phase where it JPEG-encodes each saved frame, timestamps the filename, and writes to the SD card. This design ensures the SPI frame stream is never interrupted by SD access.

Why Deferred

GPIO 13 is shared between SPI MOSI and SDMMC DATA0. On the ESP32-CAM board, these two peripherals cannot operate simultaneously. The original design tore down SPI, mounted SD, wrote a JPEG, unmounted SD, and re-initialized SPI — per frame. This cost 200-500ms per save cycle, limiting effective capture rate to ~2.5 fps.

The deferred design eliminates all mid-capture SD access. SPI runs at 18 MHz uninterrupted for the full burst duration. SD is only touched when SPI is idle.

PSRAM Save Ring Buffer

During capture, one out of every `SAVE_INTERVAL` frames (default 10) is copied into a PSRAM ring buffer for later SD write.

Memory Layout

Parameter	Value
Frame size	76,800 bytes (320x240 grayscale)
Save interval	Every 10th frame
Capture rate	30 fps
Saves per second	3
Max burst duration	~10 seconds
Max saved frames	30
PSRAM required	30 x 76.8 KB = 2.3 MB
PSRAM available	8 MB (minus ~230 KB for camera framebuffers)

Ring Buffer Structure

```
typedef struct {
    uint8_t *slots[SAVE_RING_SIZE];     // PSRAM pointers
    uint32_t timestamps[SAVE_RING_SIZE]; // elapsed microseconds from epoch
    int head;                            // next write position
    int count;                           // frames buffered
} save_ring_t;
```

- Allocated at boot with `heap_caps_malloc(MALLOC_CAP_SPIRAM)`
- `head` advances on each save, wraps at `SAVE_RING_SIZE`
- If ring is full, oldest frame is silently overwritten (circular)
- `count` tracks actual frames stored (capped at `SAVE_RING_SIZE`)

Capture-Time Save Path

In the capture loop, the save is a single `memcpy` — no encoding, no filesystem, no bus teardown:

```
if (frame_count % SAVE_INTERVAL == 0) {
    memcpy(save_ring.slots[save_ring.head], fb->buf, fb->len);
    save_ring.timestamps[save_ring.head] = esp_timer_get_time();
    save_ring.head = (save_ring.head + 1) % SAVE_RING_SIZE;
    if (save_ring.count < SAVE_RING_SIZE) save_ring.count++;
}
```

Cost: ~0.1ms for 76.8 KB PSRAM-to-PSRAM copy. Negligible versus the 33ms frame period.

Post-Capture Save Phase

After receiving STOP command from DevKitV1:

Sequence

1. **SPI deinit** — release GPIO 13
2. **SD mount** — 4-bit SDMMC at 40 MHz, single mount for entire batch
3. **For each buffered frame:**
   a. Compute absolute timestamp: `epoch + (elapsed_us / 1000000)`
   b. Format filename: `img_YYYYMMDD_HHMMSS_NNN.jpg`
   c. JPEG encode from raw grayscale (`fmt2jpg`, quality 80)
   d. Write JPEG to SD
   e. Log: filename, JPEG size, compression ratio
4. **SD unmount**
5. **Deep sleep**

Timing Budget

Step	Time	Notes
SPI deinit	~1 ms	
SD mount	~50 ms	One-time
JPEG encode (per frame)	50-100 ms	Software encoder, QVGA
SD write (per frame)	~5 ms	~15 KB JPEG at 40 MHz SDMMC
SD unmount	~10 ms	
Total (30 frames)	**~2-3 seconds**	Not time-critical

The post-capture phase is idle time — the camera has already stopped capturing and the SPI stream is finished. There is no time pressure on JPEG encoding.

Timestamp Derivation

The ESP32-CAM has no RTC. Timestamps are derived from:

1. **Epoch** — received from DevKitV1 over SPI MISO during wake handshake (see SDD120)
2. **Elapsed** — `esp_timer_get_time()` returns microseconds since boot

```
uint32_t absolute_sec = epoch + (save_ring.timestamps[i] / 1000000);
```

Drift: `esp_timer` uses the 80 MHz APB clock. Typical drift is ~10 ppm = ~0.6 seconds per minute. For a 10-second burst, drift is <0.01 seconds. Negligible for photo timestamps.

Filename Format

```
/sdcard/img_20260415_143022_001.jpg
           YYYYMMDD HHMMSS  NNN
```

`NNN` is a zero-padded sequence counter within the burst, reset to 0 on each wake cycle.

Error Handling

Condition	Behavior
PSRAM alloc fails at boot	Fatal — cannot save. Log error, disable save feature
Ring buffer full	Overwrite oldest (circular). Log warning
JPEG encode fails	Skip frame, log error, continue to next
SD mount fails	Log error, all saves lost for this burst. Frames were still transmitted over SPI
SD write fails	Log error, continue to next frame
No epoch received	Filenames use `img_unknown_NNN.jpg` fallback

Source Files

File	Purpose	Status
`src/camera_control.c`	Save ring buffer logic, post-capture loop	Needs refactor
`src/image_buffer_pool.c`	PSRAM allocation for save ring	Needs save ring extension
`src/sd_storage.c`	SD mount/write with timestamped filenames	Needs timestamp filename support

---

References

- SDD100 — ESP32-CAM Overview
- SDD102 — Capture Pipeline
- SDD120 — CAM-DevKit SPI Interface (epoch handshake)